Deep Reinforcement Learning for Traffic Light Control in Jönköping

Ben Bekir Ertugrul
School of Engineering
Jönköping University
Jönköping, Sweden

***Abstract*—This document is a model and instructions for LaTeX. This and the IEEEtran.cls file define the components of your paper [title, text, heads, etc.]. *CRITICAL: Do Not Use Symbols, Special Characters, Footnotes, or Math in Paper Title or Abstract.**

***Index Terms*—component, formatting, style, styling, insert.**

I. Introduction

A. Motivation

B. Structure of this paper

- What are SUMO limitations? - Do we train on multiple scenarios, like "rush hours" and low traffic conditions, or do we only have normal traffic? - Do we use DQN or are there also other choices like PPO (is PPO able to combine with neural nets?)?

a good starting tutorial on how to generate traffic scenarios can be found here [1].

- at the end, compare normal (static) traffic light with our solution. if possible, the static traffic light control should be decently good. maybe its possible to use a real example?

II. Background

A. SUMO

B. Proximal Policy Optimization

C. Deep Q Networks

III. Related Works

IV. Environment

A. Network

map of sweden downloaded here: https://download.geofabrik.de/europe/sweden.html geojson polygon object created here: https://geojson.io/ chosen geojson found here[1].

then, we used osmium and osmfilter to apply the polygon shape to the map of sweden. afterwards, we filter only for highways, except for those of the following types:

- footway
- path
- pedestrian
- steps
- cycleway

Fig. 1. The filter (in grey) that defines the boundaries of the road network in Jönköping.

- bridleway

finally, we use the SUMO *netconvert* tool to convert the map into a SUMO network.

as for limitations, we extract lanes including speed limit, intersections, and traffic lights. however, since we only model car traffic and not pedestrians, all pedestrian traffic lights will behave completely deterministic. this does not represent the real world where pedestrians can press a button on the traffic light to trigger a faster phase switching.

1) Demand Modeling: since there is no publicly available data regarding traffic assignment zones and demand in jönköping, we use the SUMO tool randomTrips [2] to randomly create trips. we create 3600 individual trips in total where a new trip is spawned into the simulation on every timestep.

2) Rush Hours and Noise: im not sure about this yet

B. Observations

- lane density: density is calculated on the entire lane, by considering how many cars are present in the lane and how many cars could potentially fit in the lane. https://github.com/LucasAlegre/sumo-rl/blob/main/sumo_rl/environment/traffic_signal.py#L297

[1] https://github.com/benbekir/traffic-rl/blob/main/simulations/networks/jkpg/artifacts/jkpg.geojson

[2] https://sumo.dlr.de/docs/Tools/Trip.html#randomtripspy

Fig. 2. The network as shown in the SUMO nededit UI, highlighting the traffic light placements.

- queue information: queue is calculated by counting the number of cars that are stopped at the traffic light, waiting for the green signal. https://github.com/LucasAlegre/sumo-rl/blob/main/sumo_rl/environment/traffic_signal.py#L309

C. Actions

D. Reward

- predefined reward functions: https://github.com/LucasAlegre/sumo-rl/blob/main/sumo_rl/environment/traffic_signal.py#L347

V. Experiments

VI. Results

VII. Conclusion

References

[1] D. A. Guastella, E. Montero-Porras, A. Morales-Hernández, and G. Bontempi, "Traffic modeling with sumo: a tutorial," 2025. [Online]. Available: https://arxiv.org/abs/2304.05982